Antecipe Claro — Projeto Completo

Hackathon Kanastra · Desafio Antecipe · entrega hoje (24/10/2026) às 18h Time: Ideia (produto), Radar (referências), Stack (técnica), Fluxo (UX), Forja (documento pro Codex).

---

1. Resumo da solução

Proposta de valor: a empresa sobe o CSV e, em segundos, vê quanto recebe, quais notas foram recusadas e o que fazer com cada uma.

O Antecipe Claro é um app web (Next.js com Supabase) que tem três partes:

1. **Um motor de regras guiado pelo `validadores_antecipe.json`.** Não tem nenhum valor fixo no código e só roda as regras com `ativo: true`.
2. **Preço por desconto simples**, com os parâmetros lidos do JSON: `líquido = face × (1 − taxa/100 ÷ base_dias × dias)`.
3. **A mesma validação na tela e na API**, porque as duas chamam o mesmo serviço, o `src/engine`.

Diferenciais pros jurados:

- **Recusa explicada.** Cada recusa diz qual regra falhou, o que isso significa e a próxima ação.
- **Regras editáveis pelo gestor sem deploy.** Elas são versionadas, ele vê uma prévia do impacto e depois publica.
- **Duplicidade e idempotência.** O lote é comparado com o estoque e com as cessões anteriores, e o upload aceita um `Idempotency-Key`.
- **Concentração por raiz do CNPJ do sacado**, calculada sobre o estoque do fundo somado ao lote, numa ordem determinística.
- **IA com papel limitado.** Ela só reescreve o motivo e propõe mudanças de regra, e nunca aprova nada. Se falhar, entra o texto padrão.
- **Entregáveis do desafio:** README, relatório e histórico de prompts.

2. O que os dados mostram

- **Estoque:** 100.554 notas de 18 fundos. Dessas, 10.621 são do ANTECIPE FIDC (CNPJ 92.234.400/0001-80).
- **Colunas:** fundo, cedente, sacado (nome e CNPJ), número do documento, tipo, valor nominal, vencimento e data de aquisição.
- **Formato:** separador `;`, decimal com vírgula e datas `dd/mm/aaaa`.
- **Lacunas:** o CSV não traz a situação cadastral do sacado, então ela vem de uma tabela mock (`situacao_cadastral_mock`). A data-base é configurável e começa em hoje.

Regras (do JSON)

Código	Ativa	Parâmetro
CNPJ_SACADO_VALIDO	sim	dígito verificador
SITUACAO_CADASTRAL_SACADO	sim	aceitas: ATIVA
TIPO_RECEBIVEL	sim	Duplicata, Duplicata de Serviço
VALOR_MINIMO	sim	R$ 100,00
NAO_VENCIDA	sim	vencimento > data-base
PRAZO_MAXIMO	sim	180 dias
CONCENTRACAO_MAXIMA_SACADO	sim	10% por raiz do CNPJ (8 dígitos)
PRAZO_MINIMO	não	30 dias
CONCENTRACAO_MAXIMA_CEDENTE	não	8%

Precificação: DESCONTO_SIMPLES, 2% ao mês, base de 30 dias.

3. Problema, personas e jornada

Problema. A pequena empresa vende a prazo, precisa do dinheiro hoje e não entende por que o fundo recusa parte das notas. O gestor do fundo, por sua vez, depende de deploy para mudar uma regra.

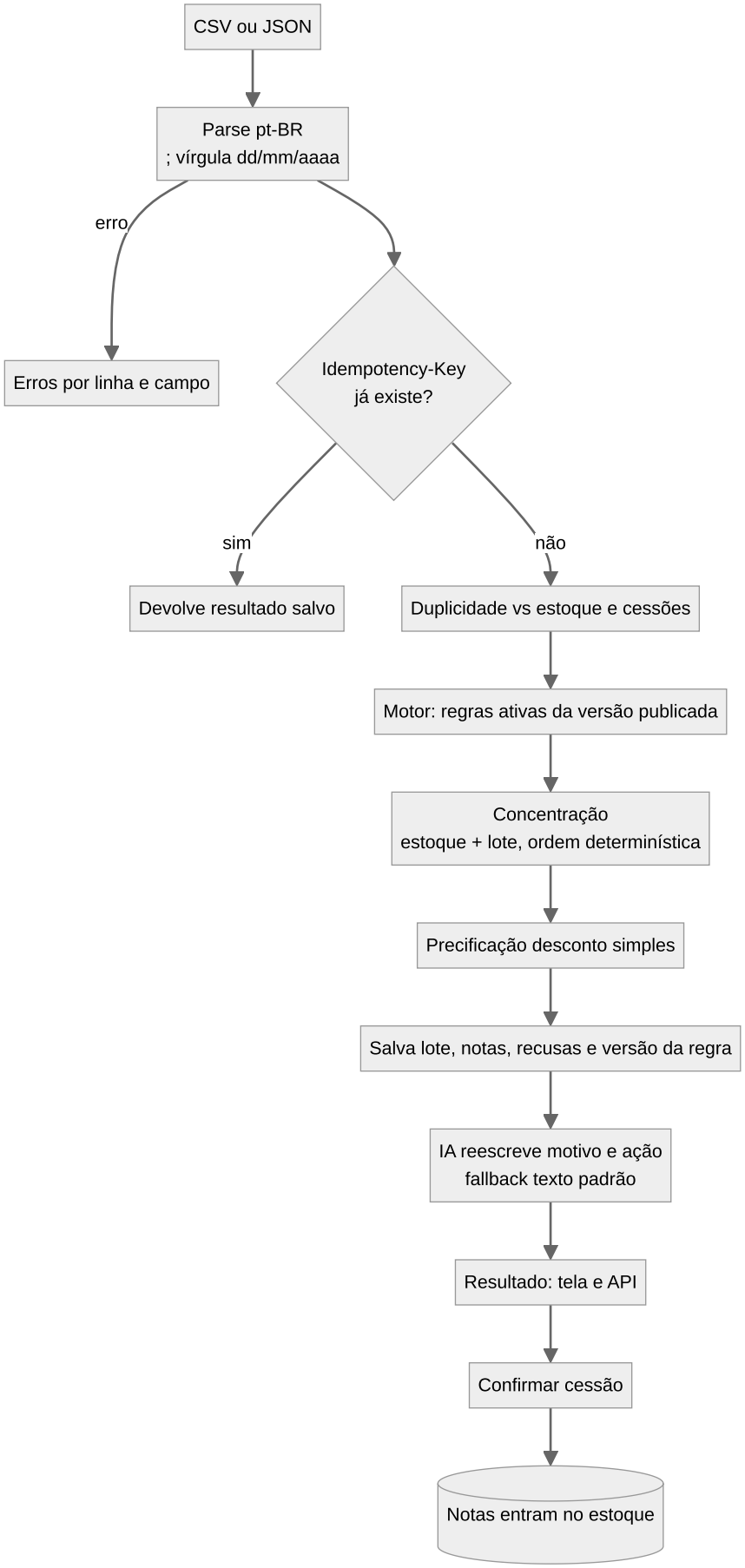
Personas

- **Carla:** dona de uma indústria de estofados, sem formação financeira.
- **Rafael:** gestor do fundo, que ajusta os validadores.

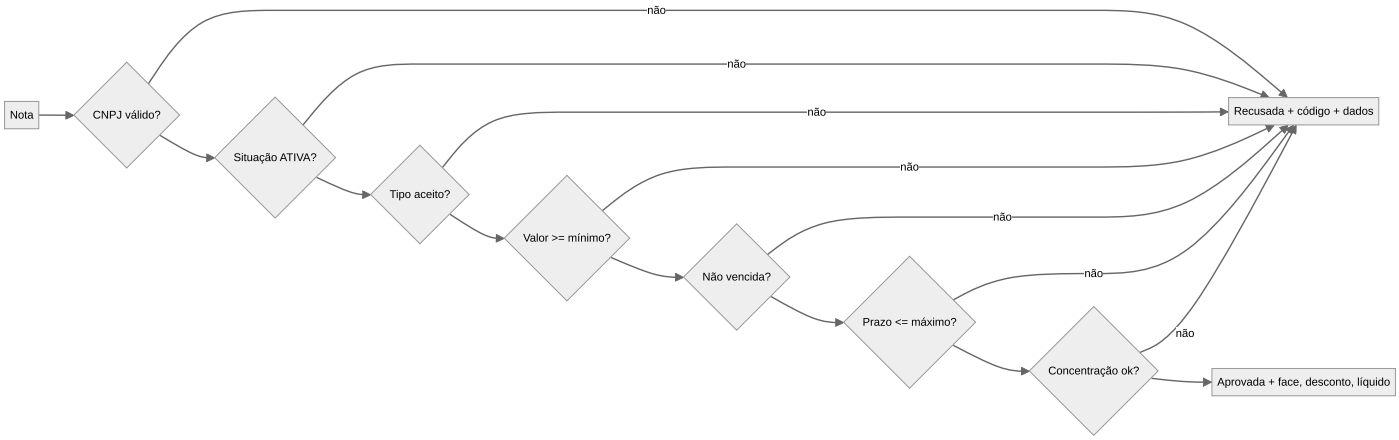
Jornada

1. A empresa sobe o CSV.
2. O app valida o formato, apontando a linha e o campo de cada erro.
3. Aparece o resumo: aprovadas, recusadas e total a receber.
4. Vem a tabela por nota.
5. Num clique, a empresa vê o motivo e a ação de cada recusa.
6. Ela confirma a cessão.
7. Do lado do gestor, ele edita uma regra, vê a prévia e publica.

4. Fluxograma da solução

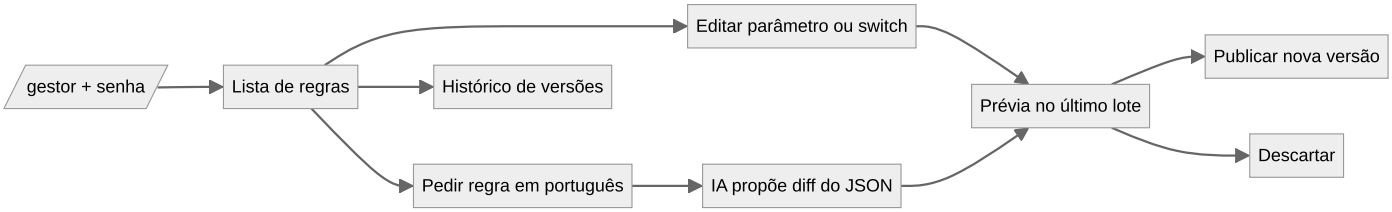


Validação de uma nota



A nota passa por **todas** as regras ativas e guarda **todos** os motivos de recusa. A tela mostra o principal primeiro.

Gestor



5. Escopo

Entra hoje

- O motor dirigido pelo JSON, com as 9 regras: as 7 ativas mais as 2 desligadas, que o gestor liga quando quiser.
- Concentração, preço, duplicidade e idempotência.
- A API e a tela.
- Regras versionadas no banco.
- A IA para explicar os motivos e propor regras.
- Um seed de demonstração com uma recusa de cada tipo.
- README, relatório e histórico de prompts.

Fica pra depois: consulta real à Receita, login com várias empresas, assinatura da cessão, registradora e pagamento.

6. Lógica e casos de borda

- **Chave de duplicidade:** `cedente + sacado + número + vencimento + valor`. É comparada com o `estoque` (ANTECIPE) e com as `cessoes`, e também dentro do próprio lote, onde a segunda ocorrência é recusada como "duplicada no arquivo".
- **Idempotência:** é feita pelo header `Idempotency-Key`. Se ele não vier, usa-se o `hash_csv` com a data-base. Reenviar o mesmo lote devolve o mesmo resultado, sem criar nada novo.
- **Concentração:**
  1. Soma o valor do estoque do fundo por raiz do CNPJ do sacado, em 1 query agrupada.
  2. Ordena o lote por vencimento e depois por número.
  3. Acumula as notas uma a uma. A nota que faria a raiz passar de `percentual_maximo` do total (estoque mais aceitas) é recusada, e o motivo mostra o percentual.
  4. O cálculo usa só as notas que passaram nas outras regras.
- **Dias:** `vencimento − data-base`, em dias corridos.
- **Valores:** calculados com `decimal.js` e arredondados em 2 casas só na exibição e na soma final.
- **Regra desconhecida no JSON:** gera um erro de configuração claro, e as outras regras continuam rodando.
- **Data inválida, número inválido ou coluna faltando:** viram erro de parse com a linha e o campo, e nada do lote é processado.
- **Lote vazio ou todas as notas recusadas:** aparece o estado vazio na tela e a confirmação fica bloqueada.
- **Confirmação da cessão:** é transacional e protegida contra duplo clique.

7. Ficha técnica (Stack)

- **App:** Next.js (App Router) com shadcn/ui, na Vercel. Tela e API ficam no mesmo repositório.
- **Banco:** Supabase (Postgres). Não há login no MVP: o `/gestor` é protegido por `GESTOR_SENHA` numa variável de ambiente, e a service role fica só no servidor.
- **Bibliotecas:** papaparse, date-fns, decimal.js, zod e Vitest.
- **IA:** Vercel AI SDK com `generateObject` e schema Zod. O provedor é o da chave que o Mateus tiver, configurado em `AI_PROVIDER` e `AI_API_KEY`.

Tabelas

- **`estoque`:** as notas do ANTECIPE, com `chave_unica` e `raiz_cnpj_sacado` (ambas com índice) e `origem` (`seed` ou `cessao`).
- **`regras_versoes`:** `id`, `versao`, `json`, `publicada_em`, `autor`.
- **`situacao_cadastral_mock`:** `cnpj` e `situacao`, com alguns casos INAPTA e BAIXADA.
- **`lotes`:** `id`, `idempotency_key` (única), `data_base`, `regras_versao_id`, `hash_csv` e os totais.
- **`notas_lote`:** `lote_id`, os campos do CSV, `chave_unica`, `status`, `valor_face`, `desconto`, `valor_liquido`, `dias`.
- **`recusas`:** `nota_id`, `codigo`, `dados jsonb`, `motivo_padrao`, `motivo_ia`, `acao`.
- **`cessoes`:** `id`, `lote_id`, `confirmada_em` e os totais.
- **`logs_ia`:** `prompt`, `resposta`, `modelo`, `criado_em`.

API

Método	Rota	O que faz
POST	/api/lotes	CSV multipart + `Idempotency-Key`
POST	/api/validar	notas em JSON, mesmo motor
POST	/api/lotes/:id/cessao	confirma cessão
GET	/api/regras	versão publicada
PUT	/api/regras	cria nova versão
POST	/api/regras/previa	roda último lote com JSON proposto
POST	/api/regras/sugerir	IA: texto → diff do JSON

Cada recusa volta como `{codigo, dados, motivo, acao}`. A API vem documentada em `openapi.json`, com exemplos de curl no README.

Pastas

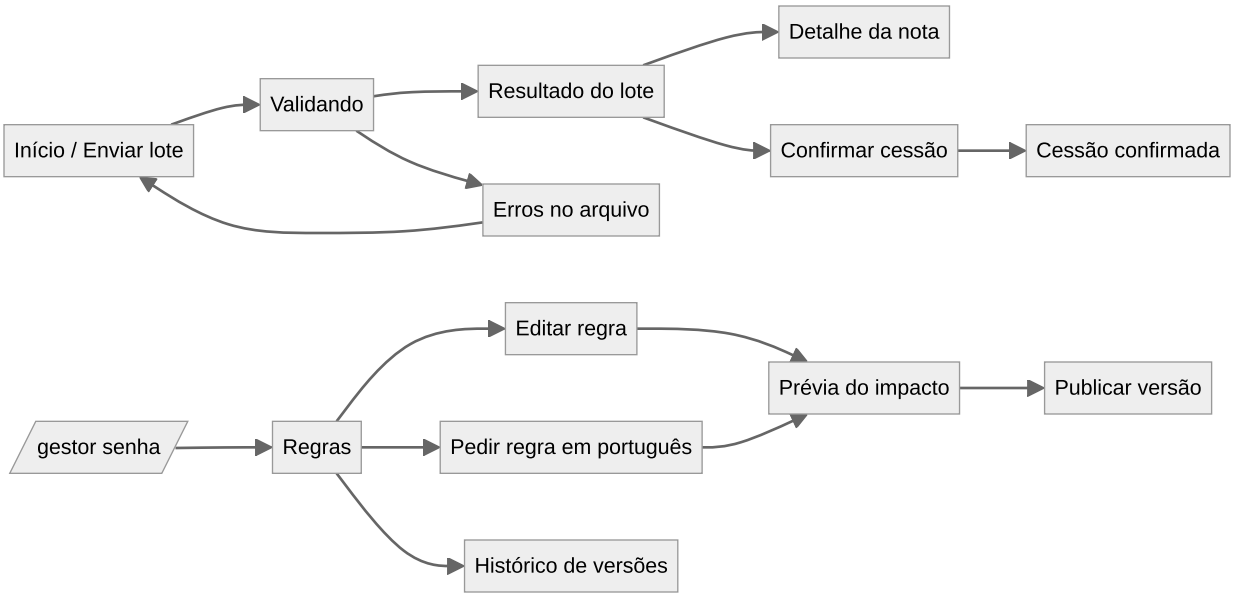
```
app/(empresa)/  app/gestor/  app/api/
src/engine/{validadores,precificacao,parse,concentracao,duplicidade}
src/ia/  supabase/migrations/  scripts/seed.ts
data/validadores_antecipe.json  data/exemplo.csv
docs/relatorio.md  docs/prompts.md  README.md  AGENTS.md
```

Custo: zero hoje, exceto a IA, porque Vercel Hobby e Supabase Free são gratuitos. O Supabase Free pausa depois de 1 semana sem uso, então vale abrir o projeto antes de os jurados testarem.

Plano B: o motor é feito só de funções puras e roda com o estoque carregado de arquivo se o Supabase travar.

8. UX (Fluxo)

Mapa de navegação



Telas (textos reais; os valores são só exemplo)

1. **Início**
   - Título "Antecipe suas notas em segundos" e subtítulo "Envie o CSV e veja na hora quanto você recebe e por que alguma nota não entrou."
   - Dropzone com "Arraste o CSV aqui ou escolha o arquivo" e a ajuda "Separador ; · decimal com vírgula · datas dd/mm/aaaa" com o link "Baixar modelo".
   - Campo Data-base, botão "Enviar lote" (desabilitado sem arquivo) e link "Usar lote de exemplo".
   - Erro: "Esse arquivo não é CSV. Envie um .csv separado por ponto e vírgula."
2. **Validando**
   - Skeleton da tabela com "Conferindo 1.000 notas com as regras do fundo...".
   - Se passar de 10 s: "Está demorando mais que o normal", com "Tentar de novo".
3. **Erros no arquivo**
   - "Encontramos 3 problemas no arquivo. Nenhuma nota foi processada."
   - Tabela com Linha, Campo e Problema.
   - Botões "Enviar arquivo corrigido" e "Baixar lista de erros".
4. **Resultado**
   - Hero "Você recebe hoje R$ X", com "Valor de face · Desconto · 2% ao mês" abaixo.
   - Chips "N aprovadas" em verde e "N recusadas" em vermelho, mais o resumo dos motivos.
   - Tabela com status, número, sacado, vencimento, dias, face, desconto e líquido. É ordenável, tem busca e pagina de 50 em 50.
   - CTA fixo "Confirmar cessão de N notas".
   - Estado vazio: "Nenhuma nota passou desta vez. Veja abaixo o que ajustar."
   - Rodapé "Regras versão 3 · data-base 24/10/2026".
5. **Detalhe da nota** (Sheet lateral). A recusa segue a ordem decisão, regra, consequência e ação:
   - PRAZO_MAXIMO: "Não entrou: essa nota vence em 240 dias e o fundo aceita até 180. Dá pra antecipar a partir de 15/03/2027."
   - CONCENTRACAO_MAXIMA_SACADO: "Não entrou: o fundo já tem 10,4% em notas da Móveis Sul (limite 10%). Tente com outro sacado ou espere vencerem notas desse sacado."
   - Duplicidade: "Essa nota já foi vendida ao fundo em 27/04/2026. Não precisa enviar de novo."
   - SITUACAO_CADASTRAL_SACADO: "Não entrou: o CNPJ do sacado está INAPTA na Receita. Confirme os dados com seu cliente."
   - TIPO_RECEBIVEL: "Não entrou: o fundo aceita só Duplicata e Duplicata de Serviço, e essa nota é Cheque."
   - VALOR_MINIMO, NAO_VENCIDA e CNPJ_SACADO_VALIDO seguem o mesmo padrão.
   - "Detalhes técnicos" recolhido, com o `codigo` e os `dados`.
   - Nota aprovada mostra o cálculo: "R$ 10.000,00 × (1 − 2% ÷ 30 × 60) = R$ 9.600,00".
6. **Confirmar cessão** (Dialog)
   - "Ceder N notas ao ANTECIPE FIDC? Você recebe R$ X."
   - Durante o envio, o botão mostra spinner e não aceita duplo clique.
   - Sucesso: "Cessão confirmada. N notas registradas."
   - Erro: "Não conseguimos confirmar. Nada foi cedido. Tente de novo."
7. **Gestor: Regras**
   - Título "Regras do ANTECIPE FIDC · versão 3 publicada em ...".

    - Cada regra tem o nome legível, o parâmetro editável, o switch Ativa e a tag "Desligada" quando estiver desligada.
8. **Prévia**
    - Comparação "Aprovadas 287 → 246 · Líquido −R$ X", com "Publicar versão 4" e "Descartar".
    - Sem lote: "Envie um lote para ver o impacto".
9. **Pedir regra**
    - Campo "Descreva a mudança, ex.: aceitar Cheque também", que leva ao diff do JSON.
    - Botões "Ver prévia" e "Descartar". A IA nunca publica sozinha.
10. **Histórico:** versão, autor, data e "Ver JSON".

Design system

- **Cores:**
    - fundo #F7F8FA, superfície #FFFFFF e borda #E3E6EB;
    - texto #0F172A e texto secundário #475569;
    - primária #1D4ED8 (hover #1E40AF);
    - aprovada #15803D sobre #DCFCE7, recusada #B91C1C sobre #FEE2E2 e atenção #B45309 sobre #FEF3C7.
- **Tipografia:** Inter. Hero 40/48 semibold com `tabular-nums`, H1 24/32, corpo 14/20 e tabela 13/18. Valores alinhados à direita, sempre no formato "R$ 1.234,56".
- **Espaçamento e forma:** grid de 4 px, raio 8 (12 em modais), sombra só em modal e painel.
- **Componentes (shadcn):** Button, Card, Badge, DataTable, Sheet, Dialog, Switch, Input, Dropzone, Skeleton, Toast e Collapsible.

Acessibilidade (WCAG AA)

- O status sempre tem ícone e texto, nunca só cor.
- Contraste de 4,5:1 ou mais.
- Tudo funciona pelo teclado, e Esc fecha o painel e devolve o foco à linha.
- O resultado é anunciado com `aria-live="polite"`.
- A tabela usa `th scope`.
- O erro de upload fica ligado ao campo com `aria-describedby`.
- Alvos de clique com 40 px ou mais, e layout responsivo a partir de 360 px (no mobile a tabela vira cards).

9. Referências (Radar)

- Simulador de deságio da Neofin: tabela por título com face, prazo, deságio e líquido, mais o resumo do lote.
- Simulador da Antecipa Fácil: "Você recebe hoje" em destaque.
- NN/g, mensagens de erro: a ordem decisão, regra, consequência e ação, sem culpar o usuário.
- **Armadilha:** esconder o motivo da recusa, como faz o crédito tradicional. Aqui as regras são públicas.

10. Pitch e métricas da demo

- **Receita:** licença por fundo. O motor pode virar um produto que a Kanastra oferece aos fundos que administra.
- **Métricas da demo:**
  - menos de 3 s do upload ao resultado com 1.000 notas;
  - 100% das recusas com motivo e ação;
  - zero duplicidade no reenvio;
  - regra mudada sem deploy.
- **Roteiro de 3 min:**
  1. Subir o CSV de exemplo e mostrar o resultado.
  2. Abrir uma recusa explicada.
  3. Reenviar o mesmo lote para mostrar a idempotência.
  4. No gestor, mudar o prazo para 120 dias, ver a prévia e publicar.
  5. Reenviar o lote com o resultado novo.
  6. Mostrar a mesma chamada por curl na API.

PARTE CODEX

Cole o arquivo `Antecipe-Codex.md` inteiro no Codex. Ele contém o AGENTS.md e as fases abaixo, e diz quais fases rodam em paralelo.

AGENTS.md (salvar na raiz do repositório)

```
# Antecipe Claro — regras fixas
- Projeto: app web pro hackathon Kanastra (desafio Antecipe). Entrega até 24/10/2026 às 18h. Priorize fazer funcionar ponta a ponta.
- Stack: Next.js App Router + TypeScript estrito, shadcn/ui, Tailwind, Supabase (Postgres), papaparse, date-fns, decimal.js, zod, Vitest e
Vercel AI SDK.
- Todo texto de UI em pt-BR. Moeda "R$ 1.234,56" e datas dd/mm/aaaa.
- NENHUM valor de regra no código: tudo vem do JSON da versão publicada (`regras_versoes`), cuja semente é `data/validadores_antecipe.json`.
- Só rodam os validadores com `ativo: true`. Um código desconhecido gera erro de configuração e não derruba o resto.
- Tela e API chamam o MESMO serviço, `src/engine`, e o motor é feito só de funções puras.
- A IA nunca aprova nem recusa nada. Ela só reescreve o motivo e a ação, e sempre tem um texto padrão de fallback.
- Dinheiro sempre com decimal.js. Nada de float.
- A service role do Supabase só no servidor. O /gestor é protegido por `GESTOR_SENHA`.
- Cada fase termina com `pnpm test` e `pnpm build` verdes.
- Registre em `docs/prompts.md` cada prompt usado (seus e da IA do app).
```

Como dividir entre agentes

- **Agente A:** Fase 0, que é a base e todos esperam.
- **Depois da Fase 0, em paralelo:**
  - Agente A: Fase 1 (motor).
  - Agente B: Fase 2 (banco e seed).
  - Agente C: Fase 4 (telas da empresa com dados mock do tipo `ResultadoLote`).
- **Depois de 1 e 2:** Agente A faz a Fase 3 (API) e o Agente B faz a Fase 5 (gestor).
- **Por último:** Fase 6 (IA), Fase 7 (entregáveis), e o Agente C integra a tela à API real.
- Os contratos de tipo em `src/engine/types.ts` (Fase 0) são a fonte da verdade entre os agentes. Ninguém muda esses contratos sem atualizar os outros.

Fase 0 — Setup e contratos

```
CONTEXTO: Antecipe Claro. A empresa sobe um CSV de notas, o app valida cada nota pelas regras do ANTECIPE FIDC (validadores_antecipe.json),
calcula o líquido e explica cada recusa. O gestor edita as regras sem deploy.
OBJETIVO: criar o projeto, os contratos de tipo e os dados de base.
STACK/REGRAS: siga o AGENTS.md.
ARQUIVOS:
- Projeto Next.js com pnpm, shadcn init, Vitest e .env.example (SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, GESTOR_SENHA, AI_PROVIDER,
AI_API_KEY).
- data/validadores_antecipe.json (cópia exata).
- data/estoque_antecipe.csv (o arquivo de 100.554 linhas, fora do git se passar de 50 MB, com instrução no README).
- src/engine/types.ts.
- AGENTS.md.
MODELO (types.ts):
  Nota {linha, nomeFundo, docFundo, nomeCedente, docCedente, nomeSacado, docSacado, numero, tipo, valorNominal: Decimal, vencimento: Date,
aquisicao?: Date}
  RegrasJSON {fundo:{nome,cnpj}, validadores:{codigo,ativo,parametros}[], precificacao:{metodo,taxa_mensal_percentual,base_dias}}
  ResultadoRegra {ok:boolean, codigo:string, dados:Record<string,unknown>}
  Recusa {codigo, dados, motivo, acao}
  NotaResultado {nota, chaveUnica, status:'aprovada'|'recusada', dias, face, desconto, liquido, recusas:Recusa[]}
  ResultadoLote {loteId, dataBase, regrasVersao, totais:{aprovadas,recusadas,face,desconto,liquido}, notas:NotaResultado[],
resumoMotivos:Record<string,number>}
  ErroParse {linha, campo, problema}
Colunas do CSV:
NOME_FUNDO;DOC_FUNDO;NOME_CEDENTE;DOC_CEDENTE;NOME_SACADO;DOC_SACADO;NU_DOCUMENTO;TIPO_RECEBIVEL;VALOR_NOMINAL;DATA_VENCIMENTO_ORIGINAL;DATA_AQUISICA
NÃO FAZER: login, telas ou lógica de negócio.
ACEITE: pnpm dev sobe, pnpm test roda e types.ts compila.
REPORTAR: a árvore de pastas e os comandos.
```

Fase 1 — Motor de regras e preço (função pura)

```
OBJETIVO: src/engine completo e testado.
ARQUIVOS: src/engine/{parse.ts, chave.ts, precificacao.ts, concentracao.ts, motor.ts, validadores/*.ts, validadores/index.ts} e testes em src/engine/__tests__.
LÓGICA:
- parse.ts: papaparse com delimiter ';' e header. Valor com vírgula vira Decimal, datas dd/mm/aaaa pelo date-fns parse com validação estrita ('31/02/2026' é erro). Devolve {notas, erros: ErroParse[]}. Se houver qualquer erro, o lote não é processado. Coluna faltando gera um erro de cabeçalho.
- chave.ts: chaveUnica = só dígitos(docCedente)|só dígitos(docSacado)|numero|vencimento ISO|valor 2 casas. raizCnpj = os 8 primeiros dígitos.
- Registro de validadores: Record<codigo, (nota, parametros, contexto) => ResultadoRegra>.
  CNPJ_SACADO_VALIDO: dígitos verificadores. dados {cnpj}.
  SITUACAO_CADASTRAL_SACADO: contexto.situacao(cnpj) contra situacoes_aceitas. Se o CNPJ não estiver no mock, conta como "NAO_ENCONTRADA" e é recusado. dados {situacao}.
  TIPO_RECEBIVEL: tipos_aceitos. dados {tipo, aceitos}.
  VALOR_MINIMO: dados {valor, minimo}.
  NAO_VENCIDA: vencimento > dataBase. dados {vencimento, dataBase}.
  PRAZO_MAXIMO: dias <= prazo_maximo_dias. dados {dias, maximo, antecipavelEm: vencimento − maximo}.
  PRAZO_MINIMO: dias >= prazo_minimo_dias. dados {dias, minimo}.
  CONCENTRACAO_MAXIMA_SACADO e CONCENTRACAO_MAXIMA_CEDENTE: rodam numa passada de lote em concentracao.ts.
- motor.ts validarLote(notas, regras, contexto{dataBase, situacao(), estoquePorRaizSacado, estoquePorCedente, totalEstoque, chavesExistentes:Map<chave,dataAquisicao>}):
  1. Duplicidade: chave já no estoque ou nas cessões gera a recusa DUPLICIDADE {vendidaEm}. Chave repetida dentro do lote gera DUPLICADA_NO_ARQUIVO {linhaOriginal}.
  2. Roda todas as regras ativas por nota e acumula todas as recusas.
  3. Concentração: só entre as notas sem recusa, ordenadas por vencimento e depois número. Acumula sobre o estoque. Recusa se (somaRaiz + face) / (totalEstoque + aceitasAteAgora + face) > percentual. dados {percentualResultante, limite, sacado}.
  4. Preço das aprovadas: desconto = face × taxa/100 ÷ base_dias × dias, líquido = face − desconto.
  5. Totais e resumoMotivos.
- Código no JSON sem validador gera um ErroConfiguracao listado no resultado, e o motor continua.
- motivoPadrao(codigo, dados) em src/engine/textos.ts, com os textos da seção UX do documento (formato decisão, regra, consequência e ação).
NÃO FAZER: acessar banco ou rede dentro do engine, ou fixar 180/10/2% no código.
ACEITE/TESTES: um teste por validador (passa e falha), parse com erro de data e de decimal, preço (R$ 10.000 com 60 dias dá R$ 9.600,00), concentração determinística (mesma entrada, mesma saída), regra inativa ignorada, código desconhecido, duplicidade no lote e no estoque.
REPORTAR: cobertura e lista de testes.
```

Fase 2 — Banco e seed

```
OBJETIVO: schema Supabase e dados carregados.
ARQUIVOS: supabase/migrations/0001_init.sql, scripts/seed.ts, src/db/*.ts.
MODELO: as tabelas estoque, regras_versoes, situacao_cadastral_mock, lotes, notas_lote, recusas, cessoes e logs_ia (campos na seção 7 do documento). Índices em estoque(chave_unica) e estoque(raiz_cnpj_sacado). lotes.idempotency_key é unique.
LÓGICA:
- O seed lê data/estoque_antecipe.csv, filtra só NOME_FUNDO = 'ANTECIPE FIDC' (são esperadas 10.621 linhas) e insere em lotes de 1.000 (ou COPY).
- Insere a versão 1 de regras_versoes com o JSON original.
- Gera situacao_cadastral_mock com todos os CNPJs de sacado como ATIVA, mais 3 INAPTA e 2 BAIXADA fixos e documentados no README.
- src/db/contexto.ts monta o contexto do motor em queries agrupadas (soma por raiz, soma por cedente, total e chaves), nunca nota por nota.
- Gera data/exemplo.csv com 30 notas novas: pelo menos uma recusa de cada regra ativa, uma duplicada do estoque, uma duplicada no arquivo e o resto aprovado.
NÃO FAZER: RLS complexa (as tabelas ficam acessíveis só pela service role).
ACEITE: o seed roda do zero, conta 10.621 e o contexto sai em menos de 500 ms.
REPORTAR: as contagens.
```

Fase 3 — API

```
OBJETIVO: as rotas da seção 7 usando src/engine.
LÓGICA:
- POST /api/lotes (multipart: arquivo, dataBase opcional, header Idempotency-Key opcional). Se a chave (ou hash do CSV + dataBase) já existe, devolve o resultado salvo com 200 e o header Idempotent-Replay: true. Senão faz parse (erros dão 422 com a lista), valida, salva e responde 201 com ResultadoLote.
- POST /api/validar: notas JSON (zod), mesma resposta, sem salvar.
- POST /api/lotes/:id/cessao: transação que cria cessoes e insere as aprovadas no estoque (origem 'cessao'). Uma segunda chamada devolve 409 "Esse lote já foi cedido".
- GET /api/regras e PUT /api/regras (exige x-gestor-senha). O PUT valida o schema e cria versao+1.
- POST /api/regras/previa: roda o último lote com o JSON proposto e devolve o antes e depois.
- openapi.json com exemplos.
ACEITE: testes de integração com o exemplo.csv. O reenvio não duplica e o PUT muda o resultado sem deploy.
REPORTAR: os exemplos de curl.
```

Fase 4 — Telas da empresa

```
OBJETIVO: as telas 1–6 da seção 8, com os textos exatos e todos os estados.
ARQUIVOS: app/(empresa)/page.tsx, app/(empresa)/lote/[id]/page.tsx e components/*.
LÓGICA: o upload chama /api/lotes com um Idempotency-Key = hash do arquivo. O resultado tem hero, chips, DataTable (busca, ordenação, 50 por página), Sheet de detalhe e Dialog de cessão. Mobile a partir de 360 px com a tabela em cards. Siga o design system e a acessibilidade da seção 8.
NÃO FAZER: inventar texto ou cor fora do documento.
ACEITE: o fluxo inteiro com o "Usar lote de exemplo", os estados vazio, erro e carregando, e navegação por teclado.
```

Fase 5 — Painel do gestor

```
OBJETIVO: as telas 7, 8 e 10 em app/gestor.
LÓGICA: login simples com GESTOR_SENHA num cookie httpOnly. A lista de regras vem da versão publicada, com nomes legíveis, parâmetro editável (número ou lista) e switch Ativa. Editar chama a prévia, que mostra "Aprovadas A → B · Líquido −R$ X". Publicar faz o PUT. O histórico tem "Ver JSON".
ACEITE: mudar o prazo máximo para 120, publicar e reenviar o lote altera o resultado, e o rodapé mostra a nova versão.
```

Fase 6 — IA (explicação e regra em português)

```
OBJETIVO: src/ia com generateObject e Zod.
LÓGICA:
- explicarRecusas(grupos): uma chamada por código distinto, com dados de exemplo. Devolve {motivo, acao} e o template é preenchido por nota.
Timeout de 5 s, e se falhar usa motivoPadrao. Salva motivo_ia.
- sugerirRegra(texto, jsonAtual) devolve o JSON proposto. A tela 9 mostra o diff e manda pra prévia. Nunca publica sozinha.
- Toda chamada é registrada em logs_ia.
NÃO FAZER: deixar a IA decidir status.
ACEITE: sem AI_API_KEY, o app funciona igual com os textos padrão.
```

Fase 7 — Entregáveis e polimento

```
- README.md: o que é, como rodar do zero (pnpm i, .env, migration, seed, dev), rotas com curl, como rodar os testes, decisões (motor guiado
pelo JSON, idempotência, concentração determinística, situação cadastral mock, data-base configurável) e limitações.
- docs/relatorio.md (1–2 páginas): problema, solução, arquitetura (o fluxograma), regras, resultados do exemplo.csv e próximos passos.
- docs/prompts.md: histórico de prompts do Codex e exportação de logs_ia (script pnpm prompts:export).
- Deploy na Vercel. Repositório público ou com acesso para os jurados.
- Medir o tempo com 1.000 notas (meta: menos de 3 s) e registrar no relatório.
```

Checklist do Mateus (antes de seguir)

- [ ] F0: `pnpm dev` sobe e o types.ts existe.
- [ ] F1: `pnpm test` verde e R$ 10.000 em 60 dias dá R$ 9.600,00.
- [ ] F2: o seed conta 10.621 notas.
- [ ] F3: o mesmo lote enviado 2 vezes não duplica.
- [ ] F4: o exemplo mostra uma recusa de cada tipo com motivo.
- [ ] F5: mudar para 120 dias muda o resultado sem deploy.
- [ ] F6: sem chave de IA tudo continua funcionando.
- [ ] F7: README roda do zero numa pasta limpa e o link do deploy abre.